

INSTITUTO FEDERAL

Mato Grosso do Sul

SSR (Server-Side Rendering)

Discentes: Bertodo Neto, Carlos Almeida, Diego Fernandes e Ueslei Garcia

Docente: Jhean Canhete

Sumário



Resumo da divisão

Pessoa	Slides	Tema	Peso
 1	1–5	Introdução + conceito + funcionamento	☆☆☆
 2	6–7, 10	Vantagens + desvantagens + desempenho	☆☆☆
 3	8–9, 11–12	SSR × SPA × PWA	☆☆☆☆
 4	13–21	Casos de uso + exemplo + conclusão	☆☆☆



Introdução

- **Server Side Rendering (SSR) :**
 - Introdução

Conceito

- **Server Side Rendering (SSR) :**
 - Conceito

Funcionamento

- **Server Side Rendering (SSR) :**
 - Funcionamento

Vantagens

- **Principais vantagens:**
 - Carregamento inicial mais rápido em muitos cenários;
 - Melhor suporte à indexação por mecanismos de busca;
 - Menor dependência do processamento do dispositivo do usuário;
 - Boa experiência em páginas que precisam exibir conteúdo rapidamente;
 - Possibilidade de gerar conteúdo personalizado no servidor.

Desvantagens

- **Principais desvantagens:**
 - Maior carga de processamento no servidor;
 - Maior complexidade de desenvolvimento;
 - Possível aumento da latência em páginas muito dinâmicas;
 - Maior necessidade de infraestrutura em aplicações com muitos acessos;
 - Algumas funcionalidades precisam ser executadas somente no cliente.

Desempenho



Comparação de SSR x SPA x PWA

Relembrando: O que é SSR

- **Server Side Rendering (SSR) :**
 - HTML é gerado no servidor e enviado pronto ao navegador.
 - Carregamento inicial rápido.
 - Bom para SEO e acessibilidade.
 - Usado em frameworks como [Next.js](#) e Nuxt.js.

Comparação de SSR x SPA x PWA

Relembrando: O que é SPA

- **Single Page Application (SPA):**
 - Carrega uma única página HTML + JavaScript.
 - Navegação acontece sem recarregar páginas.
 - Experiência fluida, mas carregamento inicial pode ser lento.
 - SEO limitado sem técnicas adicionais.

Comparação de SSR x SPA x PWA

Relembrando: O que é PWA

- **Progressive Web App (PWA):**
 - Aplicação web com recursos de app nativo.
 - Funciona offline via cache/service workers.
 - Pode ser Instalada no dispositivo.
 - SEO depende da implementação, mas o foco é experiência contínua.

Comparação de SSR x SPA x PWA

Relembrando: O que é PWA

- **Progressive Web App (PWA):**
 - Aplicação web com recursos de app nativo.
 - Funciona offline via cache/service workers.
 - Pode ser Instalada no dispositivo.
 - SEO depende da implementação, mas o foco é experiência contínua.

Comparação de SSR x SPA x PWA

Relembrando: O que é PWA

Critério	SSR	SPA	PWA
Entrega do HTML	Servidor envia pronto	Navegador Monta via JS	Similia ao SPA, com cache
Desempenho inicial	Rápido	Mais Lento	Otimizado para uso contínuo
SEO	Excelente	Limitado	Variável

Comparação de SSR x SPA x PWA

Relembrando: O que é PWA

Critério	SSR	SPA	PWA
Complexidade	Moderada/Alta	Menor	Alta(SPA + extras)
Casos ideais	Sites com SEO forte, e-commerce, blogs	Apps Interativos	Apps que precisam funcionar offline

SEO

1. SEO (Search Engine Optimization) no SSR

O SEO é um dos pontos mais fortes da arquitetura SSR (Server-Side Rendering).



Como Funciona

A página HTML é gerada inteiramente no servidor no momento da requisição. O servidor busca os dados, renderiza o HTML com as informações preenchidas e envia o documento finalizado.



Para os Crawlers

Os robôs de busca leem imediatamente o HTML pronto, com as tags `<title>`, `<meta>` e cabeçalhos `<h1>`-`<h6>` preenchidos, garantindo indexação rápida.



Redes Sociais

Ao compartilhar links, o servidor responde instantaneamente com as tags Open Graph (`og:title`, `og:image`), garantindo visualizações completas e sem falhas.

Complexidade

Embora ofereça grandes benefícios para SEO e tempo de carregamento inicial, o SSR introduz uma complexidade adicional no ciclo de desenvolvimento e infraestrutura:



Servidor 24/7

Exige um ambiente de execução ativo continuamente (como **Node.js**, Python ou PHP) para processar as requisições em tempo real.



Estado e Ciclo

Exige separar o código server-side e client-side. Objetos como **window** e **document** não existem no servidor.



Escalabilidade

Cada requisição consome CPU para renderizar HTML. Requer estratégias robustas de autoscaling, load balancing e cache de borda.



Hidratação

O cliente baixa JS para ativar interatividade. Se o HTML gerado divergir do esperado pelo cliente, ocorrem erros de mismatch.

Casos de uso ideal

O **SSR (Server-Side Rendering)** é a escolha ideal quando a aplicação exige rápida visibilidade inicial e excelente indexação em buscadores.



E-commerces

Produtos precisam de indexação rápida no Google para atrair vendas orgânicas e reter o usuário com exibição imediata.



Portais e Blogs

Conteúdo dinâmico de alto tráfego que exige indexação instantânea nas buscas e compartilhamento impecável em redes.



Marketplaces

Plataformas e classificados com anúncios que mudam constantemente e dependem fortemente de SEO local e global.



Landing Pages

Páginas institucionais de alta conversão onde o menor tempo de carregamento inicial (FCP) maximiza os resultados.



Código de Exemplo da Arquitetura SSR

Abaixo está o funcionamento prático de um servidor em **Node.js** com **Express**, ilustrando o fluxo de renderização no lado do servidor.

01. Inicialização do Express

Configuração inicial do servidor utilizando o framework Express para gerenciar as requisições HTTP.

02. Porta de Conexão

O servidor escuta na porta 3000, estabelecendo o ponto de entrada local para o fluxo de renderização.

03. Mock de Dados

Função síncrona que simula a busca de dados no banco, preparando as informações para serem injetadas no HTML.

```
1  const express = require('express');
2  const app = express();
3  const PORT = 3000;
4
5  app.listen(PORT, () => {
6    console.log('Servidor SSR rodando com sucesso em http://localhost:${PORT}');
7  });
8
9  function buscarProdutosDoBanco() {
10   return [
11     { id: 1, nome: 'Smartphone XYZ', preco: 'R$ 2.500,00' },
12     { id: 2, nome: 'Notebook Ultra', preco: 'R$ 4.800,00' },
13     { id: 3, nome: 'Fone de Ouvido Bluetooth', preco: 'R$ 350,00' }
14   ];
15 }
```



Código de Exemplo da Arquitetura SSR

Abaixo está o funcionamento prático de um servidor em **Node.js** com **Express**, ilustrando o fluxo de renderização no lado do servidor.

1. Busca de Dados Recupera os produtos diretamente do banco de dados no servidor antes de gerar a página.

2. Interpolação HTML Gera a estrutura HTML dinamicamente mesclando o layout com a lista de produtos obtida.

3. Envio ao Cliente O navegador recebe o documento HTML 100% pronto e estruturado para renderização imediata.

```
17 app.get('/', (req, res) => {
18   const produtos = buscarProdutosDoBanco();
19
20   const listaProdutosHTML = produtos
21     .map(p => `<li><strong>${p.nome}</strong> - ${p.preco}</li>`)
22     .join('');
23
24   const htmlFinal = `
25     <!DOCTYPE html>
26     <html lang="pt-BR">
27     <head>
28       <meta charset="UTF-8">
29       <meta name="viewport" content="width=device-width, initial-scale=1.0">
30       <title>Loja Virtual SSR - Catálogo</title>
31     </head>
32     <body>
33       <div>
34         <h1>Catálogo de Produtos (Renderizado no Servidor - SSR)</h1>
35         <ul>
36           ${listaProdutosHTML}
37         </ul>
38       </div>
39     </body>
40   </html>
41   `;
42
43   res.setHeader('Content-Type', 'text/html');
44   res.status(200).send(htmlFinal);
45 }
46
47
48
49 });
```



Explicação Passo a Passo do Código

Análise detalhada do fluxo de execução e responsabilidades de cada linha no servidor SSR.

Linhas 2-4 Setup Inicial

Importam o Express, iniciam a aplicação servidor e definem a porta de rede local `PORT = 3000`.

Linhas 7-13 Mock do Banco

A função `buscarProdutosDoBanco()` simula uma consulta de dados no backend antes de enviar o HTML.

Linha 16 Rota Principal

Define a rota raiz `/`. Qualquer requisição do usuário ou robô de busca dispara a execução desta rota.

Linha 18 Coleta de Dados

Chama a função simulada e armazena os dados dos produtos diretamente no ambiente do servidor.

Linhas 21-23 Mapeamento HTML

Usa o método `.map()` para converter objetos em elementos HTML `<li>` unindo-os com `.join()`.

Linhas 26-47 Template Literal

Gera o documento HTML completo e injeta dinamicamente a string com a lista de produtos gerada.

Linhas 50-51 Resposta HTTP

Define o cabeçalho como `text/html` e responde à requisição transmitindo o HTML montado.

Linhas 55-57 Ativação

O comando `app.listen` inicializa e coloca o servidor de fato para escutar conexões externas.

Conclusão



SEO & Presença

A melhor solução arquitetural para quando os requisitos primários envolvem SEO otimizado e exibição de conteúdo imediata.



Frameworks Modernos

Next.js, Nuxt.js e Remix facilitam significativamente a complexidade de gerenciar a infraestrutura ativa de servidor.



Equilíbrio Perfeito

Une com maestria a rica interatividade do ecossistema JavaScript moderno com a eficiência do HTML pré-renderizado.

Como Funciona na Prática?

01. Requisição do Cliente

O cliente (navegador ou web crawler) digita a URL e faz uma requisição **GET** /.

02. Intercepção & Consulta

O servidor intercepta essa requisição, consulta o banco de dados e obtém a lista de produtos.

03. Geração do HTML

O servidor monta a árvore HTML com os estilos e injeta os produtos na marcação.

04. Resposta do Servidor

O servidor devolve uma string HTML completa com código HTTP **200 OK**.

05. Renderização Imediata

O navegador do usuário recebe a resposta e exibe a página instantaneamente, sem a necessidade de esperar que scripts em JS adicionais façam chamadas de API (**fetch/axios**) para popular a tela.



Aplicações Reais de SSR



Casos de Sucesso

Plataformas líderes como o e-commerce **Mercado Livre** e o portal de notícias **The New York Times** utilizam SSR para garantir máxima performance.



O Desafio (SPA)

Milhões de produtos precisam ser indexados pelo Google diariamente. Com SPA pura, os buscadores falham na indexação rápida e o carregamento em redes 3G/4G é lento.



A Solução SSR

O servidor pré-renderiza a página HTML completa com dados atualizados. Isso garante indexação imediata por SEO, carregamento instantâneo e aumento real nas vendas.

Referências

MDN Web Docs

Server-side rendering (SSR)

developer.mozilla.org

Google Search Central

Understand the Web Rendering Service

Google Developers, 2023

Next.js Documentation

Pages and Layouts - SSR (Vercel, 2024)

nextjs.org/docs

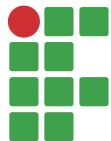
React Documentation

ReactDOMServer API & Hydration

react.dev

SLIDE DE FINALIZAÇÃO

contatos



INSTITUTO FEDERAL

Mato Grosso do Sul